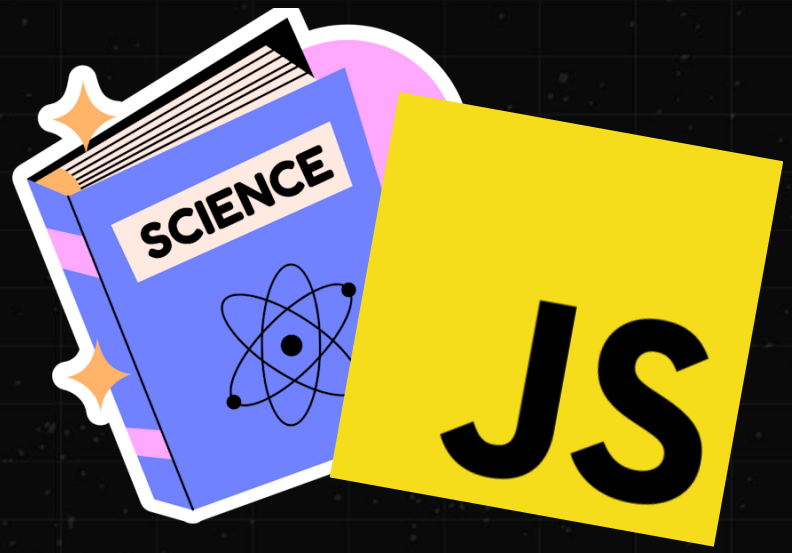
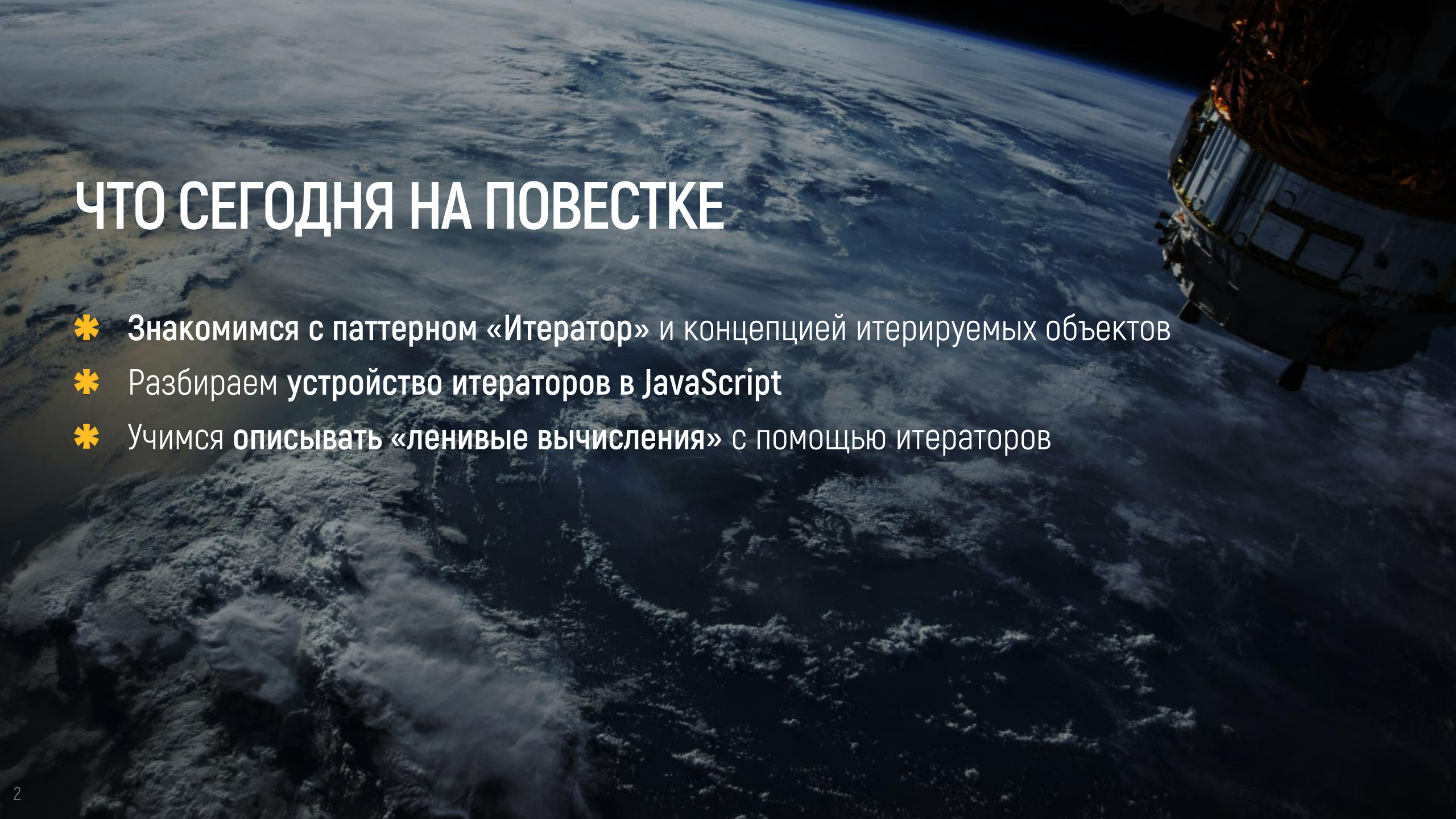
KOBEZZA.LAB

ЛЕКЦИЯ #23

CS BO FRONTEND





ЧТО СЕГОДНЯ НА ПОВЕСТКЕ

- **Знакомимся с паттерном «Итератор»** и концепцией итерируемых объектов
- Разбираем **устройство итераторов в JavaScript**
- Учимся **описывать «ленивые вычисления»** с помощью итераторов

ЧТО ЕЩЁ ЗА ИТЕРАТОРЫ?



У НАС ЕСТЬ МНОЖЕСТВО СТРУКТУР ДАННЫХ

- Массивы, матрицы, векторы
- Хэш-таблицы и различные виды деревьев
- Множество видов строк
- Все они **отличаются представлением в памяти и областью применения**
- Но почти все их **можно так или иначе обойти поэлементно в цикле**

У КАЖДОЙ СТРУКТУРЫ СВОЙ СПОСОБ ОБХОДА

- Массивы можно обойти через сдвиг индекса и цикл for
- В связных списках нужен переход по ссылке next и цикл while
- В хэш-таблицах цикл for, но нужно помнить про пустые элементы и способ разрешения коллизий
- В JavaScript для этого есть специальный цикл for in
- В строках, в зависимости от способа кодирования, **обход может отличаться**

```
// 1. Массив — сдвиг индекса
for (let i = 0; i < arr.length; i++) {
  console.log(arr[i]);
}

// 2. Связный список — переход по next
let current = head;

while (current) {
  console.log(current.value);
  current = current.next;
}

// 3. Хэш-таблица — for...in (Object)
for (const key in obj) {
  if (obj.hasOwnProperty(key)) {
    console.log(key, obj[key]);
  }
}

// 4. Строка — for...
const str = "Привет 🌍";

for (let i = 0; i < str.length; i++) {
  // Эмодзи разбивается на суррогатные пары!
  console.log(str[i]);
}
```



А НЕЛЬЗЯ ЛИ ПРИДУМАТЬ ЕДИНЫЙ ПОДХОД?

ДА

- Можно **добавить всем нашим коллекциям** метод forEach
- Каждая коллекция **обязана реализовать такой метод**
- Структура данных **может быть любой**
- Успех?

НАПРИМЕР

```
[1, 2, 3].forEach(console.log);
new Set([1, 2, 3]).forEach(console.log);
new Map([[0, 1], [1, 2], [2, 3]]).forEach(console.log);
```

МИНУСЫ forEach

- Метод принимает функцию-колбэк,
  а значит мы **не можем сделать return или break из обхода**
- Возникают **проблемы при написании асинхронного кода**
- **Использовать можно,** но это не универсальное решение

```
// 1. Нельзя прервать обход
[1, 2, 3].forEach((item) => {
  if (item === 2) break;  // ❌ Ошибка!
  // return работает как continue
});

// 2. Async/await не работает
const asyncArray = [1, 2, 3];
await asyncArray.forEach(async (item) => {
  await fetchData(item);  // ❌ Не ждёт завершения!
});
```

МОЖНО ДОРАБОТАТЬ

- Добавить поддержку всех этих механизмов можно,
  но **проблема в том, что это несовместимо с тем, что уже есть**
- Чем меньше способов сделать одну и ту же вещь – тем проще дизайн языка

А МОЖНО СДЕЛАТЬ УНИВЕРСАЛЬНЫЙ ЦИКЛ?



МОЖНО

- Давайте **сделаем объект, у которого есть специальный метод** next
- Вызов этого метода **продвигается по некоторой коллекции** на одну позицию
- Результатом next будет **элемент коллекции и флаг окончания обхода**
- Теперь мы **можем написать обычный** for для любого такого объекта
- Мы познакомились с **паттерном «Итератор»**

```
// Создаём связный список
const list = {
  value: 10,
  next: { value: 20, next: { value: 30, next: null } }
};

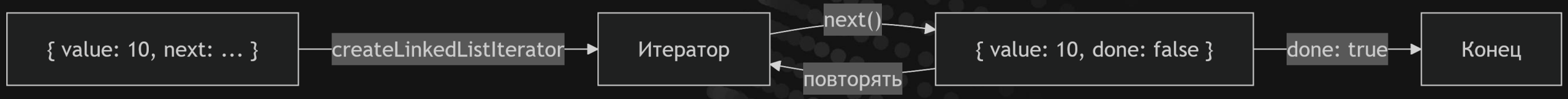
// Функция, создающая итератор для связного списка
function createLinkedListIterator(head) {
  let current = head;
  return {
    next() {
      if (current === null) {
        return { value: undefined, done: true };
      }

      const value = current.value;
      current = current.next;
      return { value, done: false };
    }
  };
}

// Использование итератора
const iterator = createLinkedListIterator(list);

for (let i = iterator.next(); !i.done; i = iterator.next()) {
  console.log(i.value);
}
```

ВИЗУАЛИЗИРУЕМ




second monitor
$0.00 (0%)
$0.00
224 DAYS TO 20
$160.00
СЛОЖНА, СЛОЖНАААА!

ВЫДЫХАЕМ

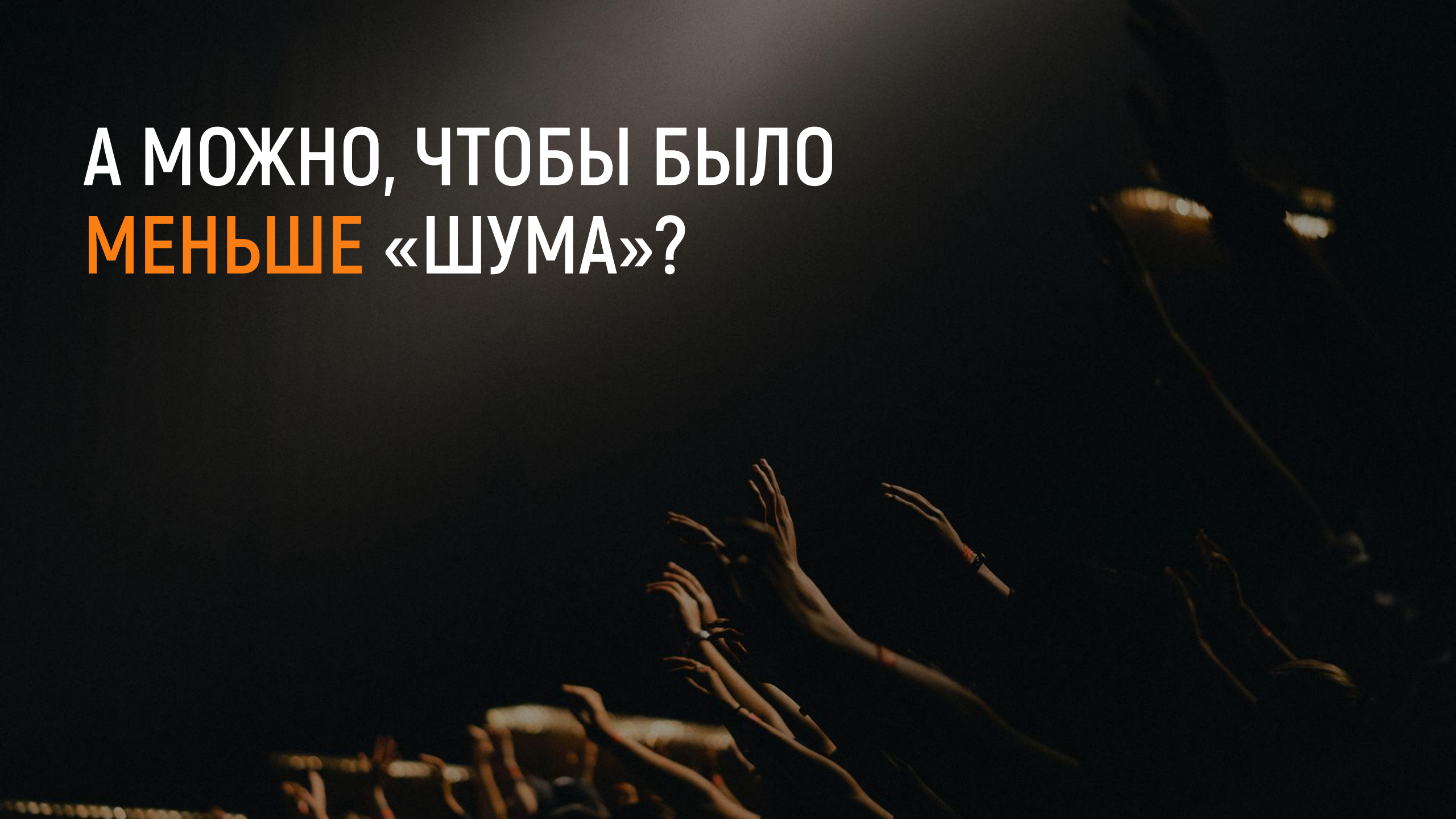
- Итератор – это просто объект, у которого есть метод next или аналогичный
- У разных коллекций будут разные реализации итератора,
  но **его интерфейс остаётся статичным**
- Использование итератора позволяет **прийти к одному универсальному циклу**
- Принимающая итератор сторона не думает про то,
  что за коллекция перед ней, но она знает как по ней итерироваться
- Так как у нас честный цикл, то работают и все остальные механизмы

```
const arr = [1, 2, 3];
const arrIterator = arr.values();

for (let i = arrIterator.next(); !i.done; i = arrIterator.next()) {
    console.log(i.value);
}

const set = new Set([1, 2, 3]);
const setIterator = set.values();

for (let i = setIterator.next(); !i.done; i = setIterator.next()) {
    console.log(i.value);
}
```


А МОЖНО, ЧТОБЫ БЫЛО
МЕНЬШЕ «ШУМА»?

СТАНДАРТИЗАЦИЯ ПОДХОДА

- Использовать обычный **for** **для такого обхода даёт много синтаксического «шума»**
- Можно было бы **добавить «синтаксический сахар»** на уровне языка
- Но для этого нужно, **чтобы используемый API был жёстко регламентирован**
- В JS стандартизация итераторов произошла в ES2015

СТАНДАРТИЗАЦИЯ ИТЕРАТОРА В JS

- Итератором может называться любой объект, у которого есть метод next
- Метод next должен возвращать объект вида { value, done }
- Введём новый синтаксис для for и spread (...) оператор

```
const set = new Set([1, 2, 3]);

const arr = [...set]; // Оператор spread использует итератор

// Обход по итератору
for (const el of set) {
   console.log(el);
}
```

ИТЕРИРУЕМЫЕ ОБЪЕКТЫ

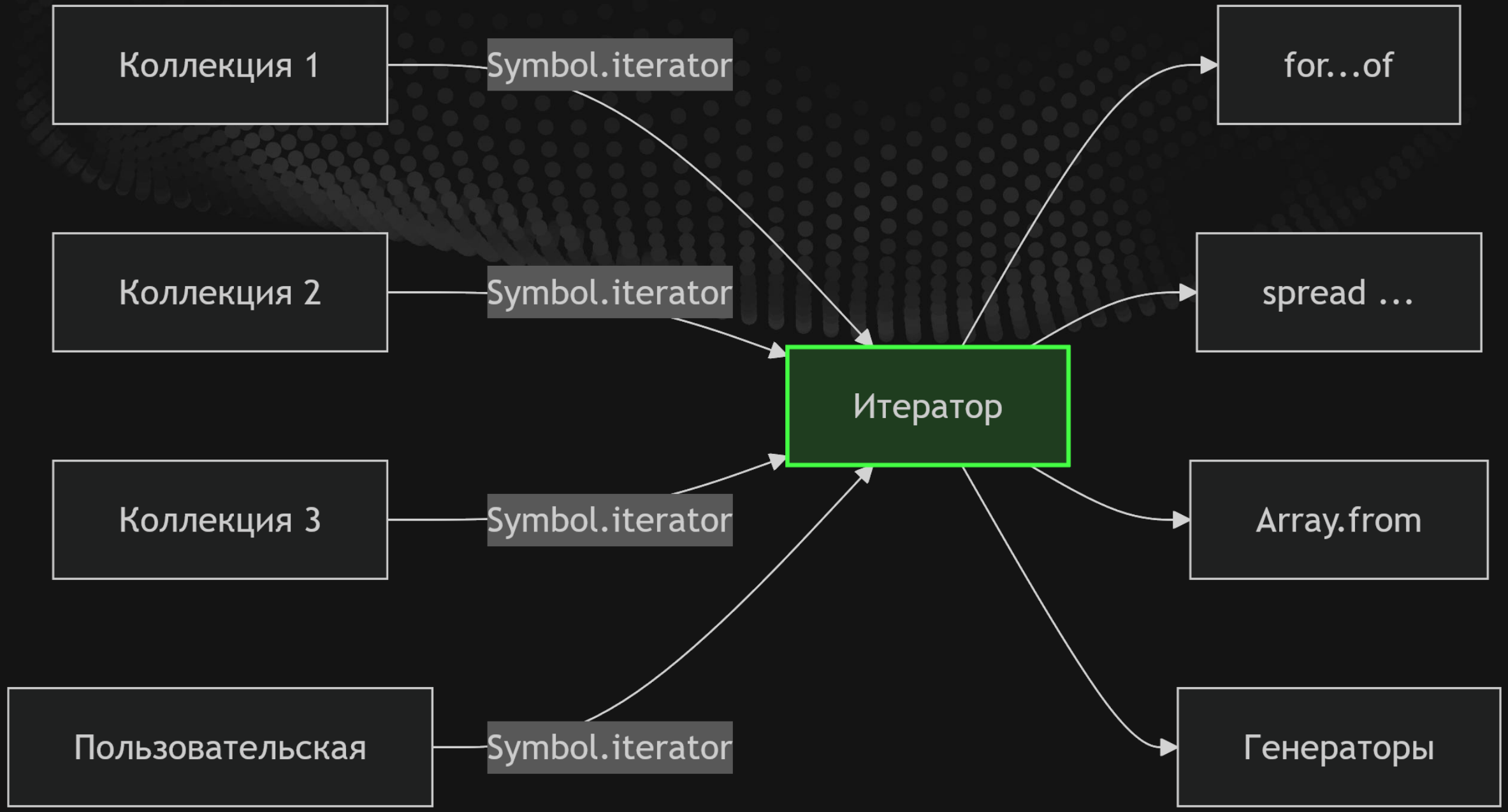
- Каждый раз писать **вызов метода, возвращающего итератор** – это много шума
- **Введём соглашение**: если у любого объекта есть метод Symbol.iterator,
  то он будет автоматически вызываться в новых конструкциях языка
- Данный метод просто **возвращает новый итератор**
- **Объекты, у которых есть такой метод,** называются итерируемыми (Iterable)

```
const set = new Set([1, 2, 3]);

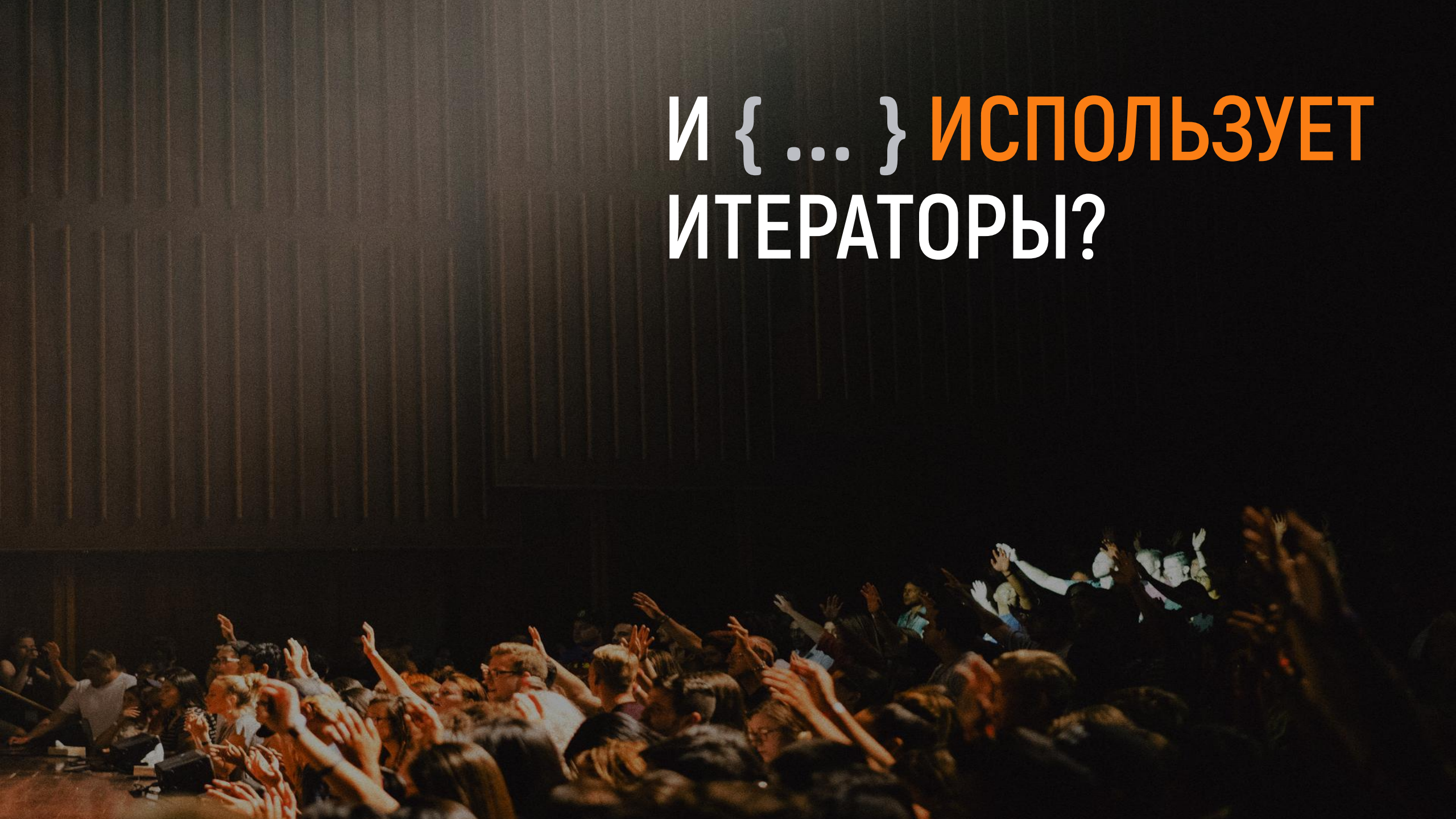
for (const el of set) {
    console.log(el);
}

// То же самое, что и выше, но явно
for (const el of set[Symbol.iterator]()) {
    console.log(el);
}

// Итератор тоже Iterable (возвращает сам себя)
for (const el of set[Symbol.iterator]()[Symbol.iterator]()) {
    console.log(el);
}
```


Коллекция 1
Коллекция 2
Коллекция 3
Пользовательская
Symbol.iterator
Symbol.iterator
Symbol.iterator
Symbol.iterator
Итератор
for...of
spread ...
Array.from
Генераторы

И { ... } ИСПОЛЬЗУЕТ ИТЕРАТОРЫ?



ДЛЯ ЛИТЕРАЛОВ { ... } ПОВЕДЕНИЕ ОТДЕЛЬНОЕ

```
const arr = [1, 2, 3, 4, 5];

// Используется итератор
console.log(...arr);

// Используется итератор
console.log([...arr]);

// Перегрузка оператора: не используется итератор
// { '0': 1, '1': 2, '2': 3, '3': 4, '4': 5 }
console.log({ ...arr });
```

```
const s = Symbol();

const obj = { a: 1, b: 2, [s]: 3 };

for (const key in obj) {
   // a 1
   // b 2
   console.log(key, obj[key]);
}

// Оператор ... копирует все enumerable свойства включая символы
const obj2 = { ...obj };

console.log(obj2[s]); // 3
```


second monitor
$0.00 (0%)
$0.00
224 DAYS TO 20
$160.00
СЛОЖНА, СЛОЖНАААА!

ВЫДЫХАЕМ

- **Поймите суть:** итератор — это некоторый объект со стандартным API для перемещения по чему угодно
- Нам **не важно, что «под капотом»** — главное, чтобы был итератор
- Итератор — это **в первую очередь очень удобный интерфейс**
- Помните про особое поведение { ... }

```
const set = new Set([1, 2, 3]);

const iter = set.values(); // Возвращает итератор

for (const el of iter) {
    console.log(el); // 1
    break;
}

// Здесь неявный вызов [Symbol.iterator]() вернет исходный итератор
for (const el of iter) {
    console.log(el); // 2
    break;
}

// Здесь неявный вызов [Symbol.iterator]() создать новый итератор
for (const el of set) {
    console.log(el); // 1 2 3
}
```

РЕФЛЕКСИРУЕМ

- Мы можем явно создать итератор, чтобы **обходить его «частями»** – нам не нужно сохранять индекс или как-то иначе запоминать позицию
- Сам итератор тоже является Iterable, но при вызове [Symbol.iterator]() **не создаётся новый итератор, а возвращается ссылка на него же**
- А вот для коллекций стандартное поведение – **создание нового независимого итератора**

```
const list = {
  value: 10,
  next: { value: 20, next: { value: 30, next: null } },
  [Symbol.iterator]() {
    return createLinkedListIterator(this);
  }
};

for (const el of list) {
  console.log(el); // 10 20 30
}

for (const el of createLinkedListIterator(list)) {
  console.log(el); // 10 20 30
}

function createLinkedListIterator(head) {
  let current = head;
  return {
    // Сделали сам итератор Iterable
    [Symbol.iterator]() { return this; },

    next() {
      if (current === null) {
        return { value: undefined, done: true };
      }

      const value = current.value;
      current = current.next;
      return { value, done: false };
    }
  };
}
```

РАЗНЫЕ ИТЕРАТОРЫ ДЛЯ ОДНОЙ КОЛЛЕКЦИИ

- Прелесть итераторов в том, что **они инкапсулируют алгоритм обхода** – нам не важно, как устроена структура, но мы знаем, как её обходить
- Но это же можно использовать для того, чтобы **обходить коллекции по-разному**
- Например, массив можно обойти с конца или в случайном порядке

```
// Итератор с конца
function reverseArrayIterator(array) {
  let index = array.length - 1;

  return {
    [Symbol.iterator]() { return this; },

    next() {
      if (index >= 0) {
        return { value: array[index--], done: false };
      }

      return { value: undefined, done: true };
    }
  };
}

// Итератор массива со случайным выбором
function randomArrayIterator(array) {
  let index = 0;

  return {
    [Symbol.iterator]() { return this; },

    next() {
      if (index < array.length) {
        index++;
        return { value: array[Math.floor(Math.random() * array.length)], done: false };
      }

      return { value: undefined, done: true };
    }
  };
}
```

```
const arr = [1, 2, 3, 4, 5];

// 5 4 3 2 1
console.log(...reverseArrayIterator(arr));

// 5 2 4 1 4
console.log(...randomArrayIterator(arr));
```

РЕФЛЕКСИРУЕМ

- Итератор по сути **регламентирует контракт обхода некоторых данных**
- Для одной и той же коллекции таких **контрактов может быть много**
- Вспомните API Intl.Segmenter

```
const text = "👩‍💻 Привет мир!";

const segmenter1 = new Intl.Segmenter("ru", { granularity: "grapheme" });
const segmenter2 = new Intl.Segmenter("ru", { granularity: "word" });

// Одна строка, но разные итераторы
console.log(...segmenter1.segment(text));
console.log(...segmenter2.segment(text));
```

РАЗНЫЕ ИТЕРАТОРЫ МОГУТ ПРОИЗВОДИТЬ РАЗНОЕ

- Например, метод массива entries **создаёт итератор по кортежам** [индекс, значение]
- А метод keys – **по индексам элементов в этом массиве**
- Таким образом, итератор не только определяет, как обходить,
  но и **что будет возвращено в результате**

```
const arr = ["a", "b", "c"];

// 1. Итератор по умолчанию — значения
for (const item of arr) {
    console.log(item); // 'a', 'b', 'c'
}

// 2. entries — кортежи [индекс, значение]
for (const [index, value] of arr.entries()) {
    console.log(index, value); // 0 'a', 1 'b', 2 'c'
}

// 3. keys — только индексы
for (const index of arr.keys()) {
    console.log(index); // 0, 1, 2
}

// 4. values — только значения (явно)
for (const value of arr.values()) {
    console.log(value); // 'a', 'b', 'c'
}
```


second monitor
$0.00 (0%)
$0.00
224 DAYS TO 20
$160.00
СЛОЖНА, СЛОЖНАААА!

ВЫДЫХАЕМ

- Итераторы — это крайне мощный примитив, который позволяет писать **выразительный, универсальный и эффективный код**
- Итераторы могут не только управлять тем, как обходить данные, но и **что мы должны получать в качестве результата**
- Это в корне отличается от подхода с явным обходом коллекции, где мы жёстко связаны с типом её элементов

РЕКУРСИВНЫЕ ИТЕРАТОРЫ

- Итераторы можно **использовать и для обхода различных рекурсивных структур**, например, деревьев или связных списков
- При реализации такого итератора можно использовать рекурсию
- Но для глубоких структур лучше **отдавать предпочтение явному стеку**

```
class BinaryNode {
  constructor(value, left = null, right = null) {
    this.value = value;
    this.left = left;
    this.right = right;
  }

  [Symbol.iterator]() {
    const stack = [this];

    return {
      next() {
        if (stack.length === 0) {
          return { value: undefined, done: true };
        }

        const node = stack.pop();

        if (node.right) stack.push(node.right);
        if (node.left) stack.push(node.left);

        return { value: node.value, done: false };
      }
    };
  }
}
```

```
const tree = new BinaryNode(1,
  new BinaryNode(2,
    new BinaryNode(4),
    new BinaryNode(5)
  ),

  new BinaryNode(3,
    new BinaryNode(6),
    new BinaryNode(7)
  )
);

for (const value of tree) {
  console.log(value); // 1, 2, 4, 5, 3, 6, 7
}
```

ОБРАТИТЕ ВНИМАНИЕ

- Итератор **выполняет работу только при вызове next**,
  а до тех пор — это просто объект с контрактом
- Кроме того, **next делает лишь минимально необходимую работу**,
  и затем итератор «приостанавливается», ожидая следующего вызова
- Такой **подход называется «ленивым»** и позволяет откладывать работу до тех пор,
  пока она действительно не понадобится

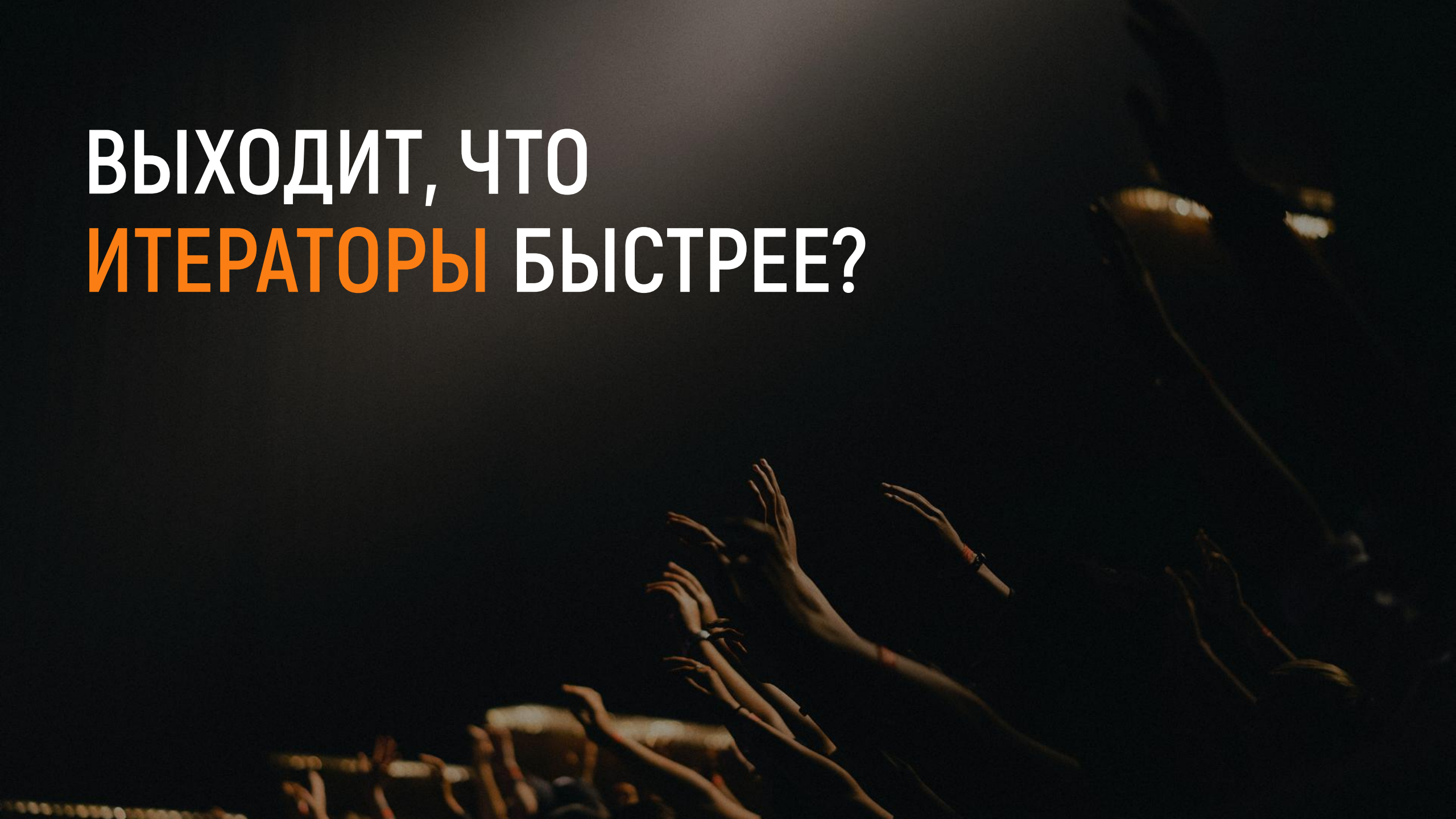
```
// Просто создается итератор и больше ничего не делается
const match = "очень большая строка".matchAll(/\d+/g);

// Запущен поиск первого вхождения, после этого итератор останавливается
console.log(match.next());

// Инициируем продолжение поиска
console.log(match.next());

// Немедленно запускается поиск всех вхождений в строке
const match2 = "очень большая строка".match(/\d+/g);

// Хоть нас интересовали только первые два вхождения,
// но работа проделана полностью
console.log(match2[0]);
console.log(match2[1]);
```


ВЫХОДИТ, ЧТО
ИТЕРАТОРЫ БЫСТРЕЕ?

ЗАВИСИТ ОТ СИТУАЦИИ

- Итераторы за счёт своей ленивой природы позволяют **не выполнять работу, которая не нужна и не используется**
- Если вам нужно найти первые 2 вхождения подстроки в большой строке или массиве, **итераторы могут оказаться драматически эффективнее**
- **Если же вы делаете «всю работу»**, то итераторы могут быть такими же, а иногда и хуже (из-за накладных расходов на объект итератора)
- Об этом поговорим на следующей лекции

ИТЕРАТОРЫ КАК ОСНОВА API

- Итераторы очень удобно использовать **как основу API функций и библиотек**
- Например, мы хотим сделать функцию split, которая возвращает **массив элементов, полученных через разбиение строки разделителем**
- Но что, если нам нужны только уникальные последовательности?
- Или только первые 3?
- Можно придумывать сложный API функции или **просто вернуть итератор**

```
function split(text, separator) {
  let start = 0;

  // Немного забегая вперёд
  return Iterator.from({
    next() {
      const index = text.indexOf(separator, start);

      if (index !== -1) {
        const value = text.slice(start, index);
        start = index + separator.length;
        return { value, done: false };
      }

      if (start < text.length) {
        const value = text.slice(start);
        start = text.length;
        return { value, done: false };
      }

      return { value: undefined, done: true };
    }
  });
}

console.log(
  new Set(split("20-10-30", "-").map(Number).take(2))
);
```

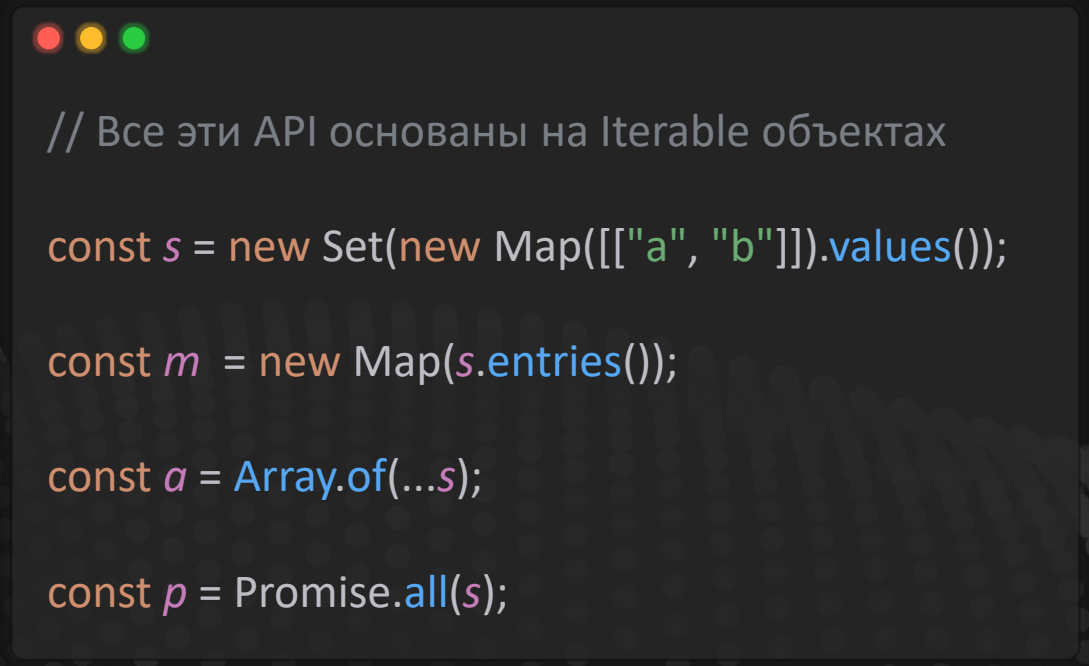

second monitor
$0.00 (0%)
$0.00
224 DAYS TO 20
$160.00
СЛОЖНА, СЛОЖНАААА!

ВЫДЫХАЕМ

- Мы **немного забежали вперёд,** но уже видно, что просто возвращая итератор мы **сделали наш API очень гибким и эффективным**
- Про методы итераторов и *Iterator*.from мы поговорим на следующей лекции
- Обратите внимание, конструктор Set **также на вход принимает итерируемый объект**
- Это значит, что множество **можно создать из любого типа данных,** для которого есть итератор

ИТЕРАТОРЫ ВЕЗДЕСУЩИ

- Они буквально **пронизывают весь современный API JavaScript** (ES2015+)
- В других языках, таких как Python или Rust, влияние итераторов ещё выше
- Какой здесь вывод? Итератор – **один из самых важных паттернов** в программировании вообще


// Все эти API основаны на Iterable объектах
const s = new Set(new Map([["a", "b"]]).values());
const m = new Map(s.entries());
const a = Array.of(...s);
const p = Promise.all(s);


Итераторы
Абстракция обхода
Скрывают детали
Единый интерфейс
Универсальный код
Ленивость
Эффективность
Композиция
Переиспользование
Безопасность
Контроль

БЕСКОНЕЧНЫЕ ИТЕРАТОРЫ

- Так как итераторы ленивы, мы можем использовать их **для создания бесконечных последовательностей**
- Это очень удобно для задач **генерации значений**
- Например, генерация случайных чисел, диапазонов и много другого

```
function range(start = 0, step = 1) {
  let current = start;

  return Iterator.from({
    next() {
      const value = current;
      current += step;
      return { value, done: false };
    }
  });
}

const iter = range(10, 2);

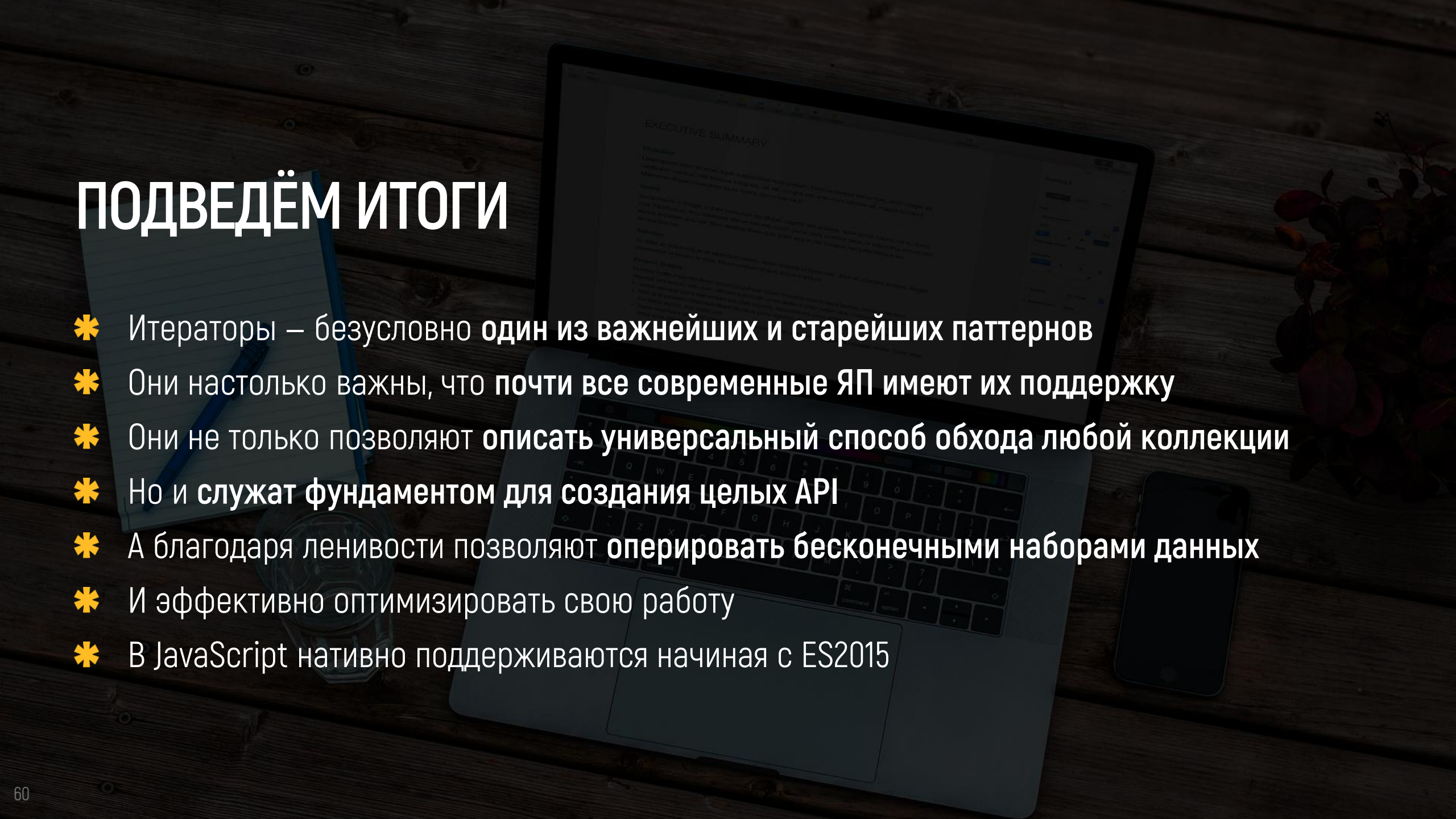
console.log(iter.next().value); // 10
console.log(iter.next().value); // 12
console.log(iter.next().value); // 14
console.log(iter.next().value); // 16

for (const value of range(0, 1).take(5)) {
  console.log(value); // 0, 1, 2, 3, 4
}
```


second monitor
$0.00 (0%)
$0.00
224 DAYS TO 20
$160.00
СЛОЖНА, СЛОЖНАААА!

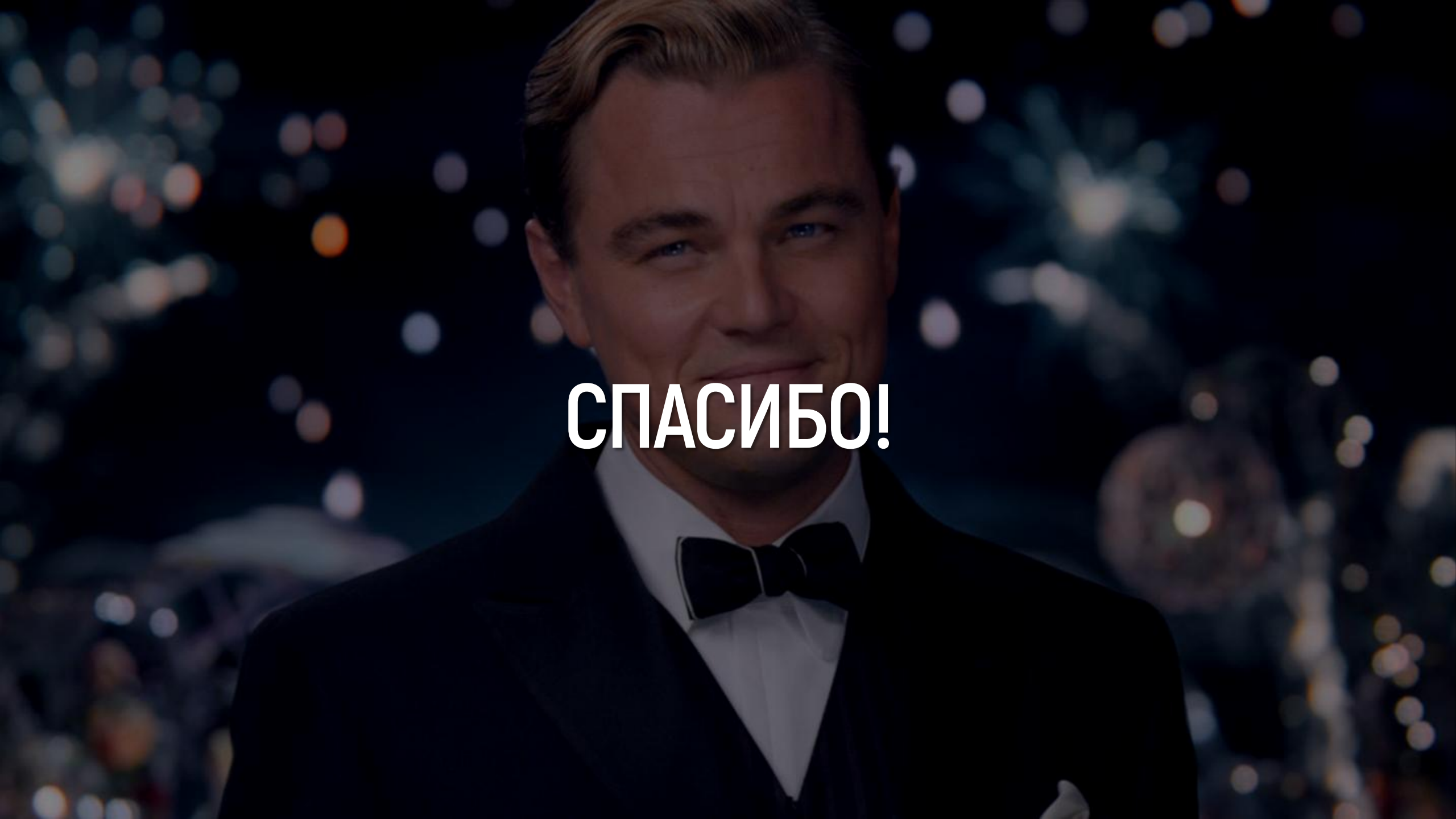
ВЫДЫХАЕМ

- На следующей лекции мы более подробно разберём кейсы использования итераторов, встроенные методы и некоторые нюансы
- Сегодня главное — понять **основную идею и реализацию в JavaScript**



ПОДВЕДЁМ ИТОГИ

- Итераторы — безусловно **один из важнейших и старейших паттернов**
- Они настолько важны, что **почти все современные ЯП имеют их поддержку**
- Они не только позволяют **описать универсальный способ обхода любой коллекции**
- Но и **служат фундаментом для создания целых API**
- А благодаря ленивости позволяют **оперировать бесконечными наборами данных**
- И эффективно оптимизировать свою работу
- В JavaScript нативно поддерживаются начиная с ES2015


СПАСИБО!